Scope Alignment & Feature Validation

Project: Company RAG Chatbot: a multi-tenant, bilingual retrieval-augmented assistant for company knowledge
Author: [YOUR FULL NAME] Supervisor: [SUPERVISOR NAME] Date: [MONTH YEAR]

How to read this document

Every feature below was planned in the approved project description, the roadmap, or the supervisor feedback. **Completed** = implemented and covered by evidence; **Partial** = implemented in part, with the gap stated; **Not implemented** = not done. Evidence points to files in the repository (paths are relative to the repository root) and to the automated tests that exercise the feature. Screenshots of each screen are in docs/screenshots and are shown in the walkthrough video.

Summary: **30 completed**, **4 partial**, **2 not implemented** out of 36 planned features.

Feature validation table

Planned feature	Intended purpose	Status	Evidence (repository reference)
Global RAG configuration (documents, cleaning, chunking, embedding, retrieval, LLM, prompts, messages, chat limit)	One central, adjustable RAG setup for every company without code changes	**Completed**	backend/app/routers/superadmin.py, backend/app/schemas.py; UI frontend/src/app/superadmin/rag; backend/tests/smoke.py 'config update + activation'
Safe configuration changes (live validation, last working config kept, version history, rollback)	A bad key or model name must never break all companies	**Completed**	backend/app/services/config_service.py; backend/tests/smoke.py 'invalid config rejected', 'NOT activated', 'previous config still active'
API credential management (add / edit / replace / delete, encrypted, never sent to browser)	Protect secrets while letting the admin change providers	**Completed**	backend/app/security.py (Fernet), backend/app/routers/superadmin.py; backend/tests/smoke.py 'key masked', 'never returns key'
Company management (register, view, activate/deactivate, enter as admin, per-company details)	Operate the multi-tenant platform	**Completed**	backend/app/routers/superadmin.py; frontend/src/app/superadmin/page.tsx, frontend/src/app/superadmin/companies/[id]; backend/tests/smoke.py
Evaluation settings (ask every N answers; 1-5 rating and True/False questions)	Collect answer-quality feedback in a controlled way	**Completed**	backend/app/routers/superadmin.py; frontend/src/app/superadmin/evaluation; backend/tests/smoke.py 'evaluation requested'
Platform and per-company analytics; reported cases	Monitor usage, quality, errors and complaints	**Completed**	backend/app/services/analytics.py; frontend/src/app/superadmin/analytics; backend/tests/smoke.py 'platform analytics'
Employee management (create, edit, role, status, password reset)	Let each company control who can use the assistant	**Completed**	backend/app/routers/company_admin.py; frontend/src/app/admin/employees; backend/tests/smoke.py 'employee created'
Document upload and processing pipeline (PDF, DOCX, TXT, MD; adjustable limits; progress; retry; EDA statistics)	Turn company files into a searchable knowledge base and show readiness	**Completed**	backend/app/services/ingest.py, backend/app/rag/*; frontend/src/app/admin/documents; backend/tests/smoke.py 'document processed', 'EDA stats stored'. Legacy .doc not supported
Chunking strategies incl. parent-child (plus recursive and fixed)	Better retrieval: search small chunks, answer with larger context	**Completed**	backend/app/rag/chunking.py; backend/tests/test_language.py (4 chunking tests)
Hybrid retrieval: dense vector + sparse keyword search fused with RRF	Combine semantic and exact-term matching	**Completed**	backend/app/rag/retrieval.py; db/schema.sql (HNSW + GIN indexes); benchmark Sec. 5
Reranking model and LLM-based query transformation (stated as 'later' in the proposal)	Higher answer precision	**Partial**	Extension slots and a follow-up query transform exist (backend/app/rag/retrieval.py); no reranking model implemented
Start / Pause / End service with complete data deletion (tracked, retried)	Control usage and guarantee data removal	**Completed**	backend/app/services/deletion.py, backend/app/routers/company_admin.py; backend/tests/smoke.py 'deletion completed', v11 'deletion completed incl. anonymise'
Company analytics: feedback with question and answer, reports, frequent questions (count, info found), conversations, CSV downloads	Judge answer quality and see knowledge gaps	**Completed**	backend/app/services/analytics.py; frontend/src/app/admin/analytics; backend/tests/smoke.py 'questions CSV download', 'conversation export'
Chat with follow-ups; maximum nine exchanges per chat enforced in the backend	Controlled conversations and cost	**Completed**	backend/app/services/chat.py; db CHECK; backend/tests/smoke.py '10th question blocked by backend'
ChatGPT-style interface (New chat, chat history, Report)	Familiar, easy-to-use employee experience	**Completed**	frontend/src/app/chat/page.tsx
Processing indicator while an answer is prepared	Progress feedback without exposing model reasoning	**Partial**	Timed two-stage indicator in frontend/src/app/chat/page.tsx; not driven by real server progress (no streaming)
Rate answers 1-5 / True-False and report issues or recommendations	Quality feedback loop	**Completed**	frontend/src/app/chat/page.tsx; backend/app/routers/chat.py; backend/tests/smoke.py 'feedback stored', 'report submitted'
Grounded answers, no-evidence behaviour, retrieval traceability	Avoid invented company information; let admins audit answers	**Completed**	backend/app/services/chat.py, retrieval_logs table; backend/tests/smoke.py 'no-evidence question uses no-evidence path'
Distinct user-facing messages for AI failure, rate limit, document-processing failure	Clear errors while technical detail stays in protected logs	**Completed**	backend/app/schemas.py (MessagesCfg), backend/app/services/common.py (error_logs)
Background workers for ingestion and deletion	Long jobs must not block requests	**Completed**	backend/app/worker.py (PostgreSQL queue, FOR UPDATE SKIP LOCKED); backend/tests/smoke.py (async ingestion)

Planned feature	Intended purpose	Status	Evidence (repository reference)
Multi-tenant isolation (company_id on every record, backend scoping, row-level security)	No company can ever see another company's data	**Completed**	db/schema.sql (RLS); backend/app/deps.py; backend/tests/smoke.py 'cannot retrieve company A's documents'; RLS test Sec. 4
Authentication, role-based access, Argon2 password hashing, audit logs	Protect accounts and administrative actions	**Completed**	backend/app/security.py, backend/app/deps.py, audit_logs; backend/tests/smoke_v11.py 'applicant cannot call admin / superadmin / chat APIs'
Backup and log-retention policy consistent with deletion	Deleted data must not linger	**Partial**	Policy stated in the privacy page; backups depend on the hosting provider; nothing implemented in code
English + Kinyarwanda: cleaning, chunking, embedding, retrieval, prompts	Serve bilingual companies; match a question in one language to a document in the other	**Completed**	backend/app/rag/lang.py, chunking.py, clean.py, ai.py; Languages tab + live 'Language check'; backend/tests/test_language.py, smoke_v11.py (Kinyarwanda tests)
Company request and review workflow (apply, approve / reject / request info, replies by portal and email)	Replace manual company registration	**Completed**	backend/app/routers/apply.py, applications_admin.py; frontend/src/app/apply, frontend/src/app/superadmin/applications; backend/tests/smoke_v11.py (registration, review, reply, reject, reapply, approve, onboarding checks). Email is outbound only and needs SMTP
Manual verification only (no automatic legitimacy checks)	Keep access open to small businesses	**Completed**	By design: the reviewer decides in backend/app/routers/applications_admin.py
Privacy and terms onboarding (3 pages, versioned acceptance), data export before ending, anonymised feedback, storage disclosure, employee-training advice	Informed consent and data protection	**Completed**	backend/app/routers/apply.py, cms.py, company_admin.py (/export); backend/tests/smoke_v11.py 'terms', 'export', 'anonymised'
Marketing site and DB-managed content (landing page, onboarding pages, form fields, languages)	Attract customers; change content without code	**Completed**	frontend/src/app/page.tsx; backend/app/routers/cms.py; backend/tests/smoke_v11.py (CMS checks)
API documentation and tooling (Swagger, Postman collection)	Testability and integration	**Completed**	/docs (Swagger), postman/company-rag.postman_collection.json (109 requests)
Model evaluation lab (SuperAdmin): retrieval/embedding and LLM metrics, baselines, threshold analysis	Measure each AI part separately and tune it with evidence	**Completed**	backend/app/eval/*, backend/app/routers/model_eval.py; frontend/src/app/superadmin/model-eval; backend/tests/test_eval_metrics.py (9), smoke_eval.py (40); docs/MODEL_EVALUATION_METRICS.md. Real-Gemini runs still to be recorded
Favicon and app icons	Professional, recognisable product in browser tabs and home screens	**Completed**	frontend/src/app/app/icon.svg, favicon.ico, apple-icon.png; browser test 'favicon asset is served'
Automated testing (unit, integration, REST, browser, load)	Validate behaviour and find defects	**Completed**	267 automated checks, 0 failing: unit 39, integration 167, REST 32, browser 29; API coverage 84%. docs/TESTING_REPORT.md, docs/testing/logs
Acceptance testing with 20+ participants (SUS)	Show real users can complete tasks and rate usability	**Partial**	Questionnaire, response template and scoring script ready (docs/testing/acceptance); no participants tested yet
Local deployment with Docker	Reproducible setup	**Completed**	docker-compose.yml, RUN_LOCALLY.md
Cloud deployment: backend on Render, frontend on Vercel	Public availability	**Not implemented**	Hosting on Render/Vercel not done yet. Deployment guide prepared; production mode and managed-database compatibility verified locally (ENVIRONMENT=production, non-superuser owner, full test suites passed). Update after deploying.
Billing per usage; image/screenshot understanding; marketing emails	Declared as future work by the supervisor	**Not implemented**	Out of scope for this version

Scope changes and justification

Change	What changed	Justification
Company registration	Manual registration by the SuperAdmin became an online request-and-review workflow.	Supervisor feedback (points 2, 3): scales onboarding and keeps manual verification.
Company is created at terms acceptance	The company record is created when the applicant accepts the terms, not at approval.	No workspace exists before consent to the terms.
Job queue	A PostgreSQL queue (FOR UPDATE SKIP LOCKED) replaced a separate Redis/Celery broker.	One fewer service to run and pay for; sufficient for the expected load.
Document types	DOCX only (no legacy .doc).	Reliable text extraction with a pure-Python library.
Embeddings	768-dimension vectors; gemini-embedding-2 supported (no task_type, prefixes instead).	pgvector HNSW indexes are limited to 2000 dimensions; the model is multilingual.
Chat limit	Nine exchanges (1 question + 8 follow-ups), configurable down to 1.	Reconciles the project description (9) and the roadmap (8).
End of service	Employee accounts are deleted too; only de-identified scores are kept.	Privacy decision and supervisor point 4.
Processing indicator	Client-side timed stages instead of server streaming.	Satisfies the requirement at far lower complexity; streaming listed as future work.

Gaps that remain

- **Reranking model and LLM-based query transformation (stated as 'later' in the proposal)** (Partial): Extension slots and a follow-up query transform exist (backend/app/rag/retrieval.py); no reranking model implemented

- **Processing indicator while an answer is prepared** (Partial): Timed two-stage indicator in frontend/src/app/chat/page.tsx; not driven by real server progress (no streaming)
- **Backup and log-retention policy consistent with deletion** (Partial): Policy stated in the privacy page; backups depend on the hosting provider; nothing implemented in code
- **Acceptance testing with 20+ participants (SUS)** (Partial): Questionnaire, response template and scoring script ready (docs/testing/acceptance); no participants tested yet
- **Cloud deployment: backend on Render, frontend on Vercel** (Not implemented): Hosting on Render/Vercel not done yet. Deployment guide prepared; production mode and managed-database compatibility verified locally (ENVIRONMENT=production, non-superuser owner, full test suites passed). Update after deploying.
- **Billing per usage; image/screenshot understanding; marketing emails** (Not implemented): Out of scope for this version